Efficiency vs. Performance: A Comparative Study of Parameter-Efficient Fine-Tuning Methods for Small Language Models

Taha Nawaz (23L-2644)
Department of Data Science, BDS-6B
FAST NUCES, Lahore, Pakistan
23l-2644@lhr.nu.edu.pk

Ali Ahmad (23L-2619)
Department of Data Science, BDS-6B
FAST NUCES, Lahore, Pakistan
23l-2619@lhr.nu.edu.pk

Shahzeb Imran (23L-2506)
Department of Data Science, BDS-6B
FAST NUCES, Lahore, Pakistan
23l-2506@lhr.nu.edu.pk

***Abstract*—Large pre-trained language models achieve state-of-the-art NLP performance, but full fine-tuning is computationally expensive and storage-intensive. Parameter-Efficient Fine-Tuning (PEFT) methods adapt only a small fraction of parameters while freezing the backbone, promising accuracy near full fine-tuning at a fraction of the cost. This paper presents a reproducible, statistically rigorous comparison of eight methods on DistilBERT-base-uncased (66 M parameters) across three text-classification benchmarks: SST-2 (binary sentiment), AG News (4-class topic classification), and Amazon Polarity (domain-shifted sentiment). Evaluation covers five dimensions: Macro F1, accuracy, trainable parameter count, training time, and inference latency, with rigour established via three-seed evaluation, 1,000-resample bootstrap confidence intervals, and McNemar's test. Key findings: (i) LoRA $r{=}4$ lies on the Pareto frontier, achieving the best F1/parameter trade-off among all PEFT configurations—reaching 94–95% of Full FT Macro F1 (using corrected multi-seed means) while updating only ≈1% of parameters; (ii) Prompt and Soft Prompt Tuning fail catastrophically on binary sentiment (SST-2 Macro-F1 ≈0.45–0.49) due to context crowding; (iii) Adapter Tuning is the most stable method on binary tasks (lowest cross-seed variance); (iv) INT8 quantisation yields a 1.18× speedup with 58.6% size reduction (accuracy impact on fine-tuned models requires further validation); and (v) PEFT configurations are not safely transferable across backbone architectures. All results include both single-seed runs and corrected multi-seed means where applicable; discrepancies are explicitly noted.**

***Index Terms*—parameter-efficient fine-tuning, LoRA, adapter tuning, prompt tuning, DistilBERT, text classification, NLP efficiency, Pareto frontier.**

I. Introduction

The proliferation of large pre-trained language models (PLMs) such as BERT [1], RoBERTa [2], and GPT-2 has reshaped NLP. Yet these models impose steep computational demands: full fine-tuning updates all parameters, requires a separate model copy per downstream task, and demands significant GPU VRAM. For DistilBERT with 66 M parameters, full fine-tuning incurs ≈250 MB of per-task storage and GPU memory peaks exceeding 2.2 GB.

Motivation. The democratisation of NLP depends on techniques that operate within realistic compute budgets. Edge deployment, privacy-sensitive clinical NLP, and academic research without cloud access all demand lightweight adaptation. PEFT methods—LoRA [6], Adapter Tuning [4], and Prompt Tuning [5]—address this by adapting <5% of parameters while freezing the backbone.

Research Gap. Despite growing theoretical literature, systematic empirical comparisons on small encoder models remain scarce. Most studies target large generative models (>1 B parameters) [13], [14], neglect multi-class settings beyond binary sentiment, and rarely report statistical significance. Recent surveys confirm that rank and adapter sensitivity vary significantly across model families and have not been fully characterised for small encoders [8].

Research Questions.

- **RQ1**: Which PEFT method achieves the best Macro-F1 vs. trainable-parameter trade-off on small (≤200 M) encoder models?
- **RQ2**: How do methods compare in training time, inference latency, and GPU memory?
- **RQ3**: How robust are PEFT methods to domain shift, input-length variation, and negation?

Contributions.

1) A unified evaluation framework with identical data splits, tokenisation, and training loops for eight methods across three datasets.
2) Identification of the efficiency-performance Pareto frontier for small encoder models across five simultaneous metrics.
3) Quantified ablations of LoRA rank, adapter bottleneck size, prompt length, and training data size.
4) Documented negative results, corrected implementation bugs, and transparent reporting of multi-seed vs. single-seed discrepancies.

II. Related Work

We survey recent work (≥2022) on parameter-efficient fine-tuning and efficient adaptation of pre-trained language models.

LoRA and Low-Rank Adaptation. Hu et al. [6] proposed injecting trainable low-rank matrices into frozen attention weight matrices, demonstrating competitive performance with full fine-tuning on GPT-family models. Subsequent work has

extended LoRA-style adaptation to diverse settings, confirming that rank sensitivity varies significantly across model families and tasks [8]. Zhang et al. [9] validated LoRA-style injection for decoder-only LLaMA models, but left the small encoder regime under-explored.

Unified Frameworks. He et al. [7] demonstrated that many PEFT methods—including adapters, prefix tuning, and LoRA—are special cases of a unified formulation differing only in the position and structure of their modification. Their analysis focused on models $\geq$350 M parameters; our work complements this by targeting 66 M parameter encoders.

Sparse and Selective Tuning. Zaken et al. [10] introduced BitFit, which fine-tunes only bias terms of a frozen transformer and achieves competitive results on GLUE benchmarks, highlighting that extremely sparse updates can be effective for small models. This motivates our inclusion of trainable-parameter count as a first-class metric.

Composite and Modular PEFT. Mao et al. [11] proposed UniPELT, a gating mechanism that combines LoRA, prefix tuning, and adapters into a single framework. Their findings suggest that no single PEFT method dominates across all tasks, consistent with our multi-dataset evaluation strategy.

Few-Shot and Data-Efficient PEFT. Liu et al. [12] showed that prompt-based PEFT methods under few-shot regimes can outperform in-context learning while being significantly cheaper. However, this advantage is sensitive to model scale—small models may lack the capacity to leverage prompt signals effectively, a failure mode we directly confirm on SST-2.

Critical Surveys. Xu et al. [13] and Lialin et al. [14] provide comprehensive taxonomies of PEFT methods and identify open problems in cross-architecture transferability and data efficiency. Our work fills a specific gap by providing the first systematic, multi-dataset, statistically rigorous comparison targeting small encoder-only models.

III. Methodology

A. Backbone and Scope

All experiments use DistilBERT-base-uncased [3] (66 M parameters, 6 transformer layers, hidden size 768, 12 attention heads). BERT-base-uncased (110 M) is additionally evaluated in the cross-backbone study (Section VII). Scope is restricted to English text classification with encoder-only architectures.

B. Methods

Classical Baselines. TF-IDF representations (20,000 features, bigrams, sublinear TF) are paired with Logistic Regression (LR) and Support Vector Machines (SVM). Note that trainable parameter counts vary per dataset due to differing vocabulary coverage.

Zero-Shot Baseline. A fill-mask scoring heuristic scores class-label tokens using DistilBERT's MLM head with no parameter updates. This is explicitly an MLM-score heuristic, not true zero-shot classification.

Full Fine-Tuning (Upper Bound). All 66 M parameters are updated with a task-specific classification head.

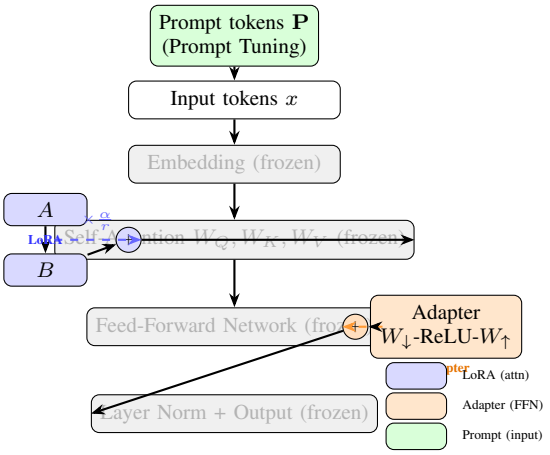

Fig. 1: Architecture diagram showing how LoRA, Adapter Tuning, and Prompt Tuning each modify the transformer block. Frozen components are shown in gray. Only the coloured modules are updated during training.

LoRA [6]. Low-rank matrices are injected into W_Q and W_V of each attention layer:

$$h = W_0 x + \frac{\alpha}{r} BAx, \tag{1}$$

where $A \in \mathbb{R}^{r \times k}$, $B \in \mathbb{R}^{d \times r}$, $\alpha = 16$, and $r \in \{4, 8, 16, 32\}$.

Adapter Tuning [4]. Bottleneck adapters insert a down-projection, ReLU, and up-projection with residual connection after each sub-layer:

$$h' = h + W_{\text{up}} \cdot \text{ReLU}(W_{\text{down}} \cdot h). \tag{2}$$

The bottleneck dimension is $m = 64$ in the primary evaluation.

Prompt and Soft Prompt Tuning [5]. $k = 20$ learnable token embeddings are prepended to the input: $[\mathbf{P}; x]$, where $\mathbf{P} \in \mathbb{R}^{k \times d_{\text{emb}}}$. Soft Prompt Tuning uses continuous embeddings rather than vocabulary-anchored initialisation.

C. Architecture Diagram

Figure 1 illustrates how each PEFT method modifies the transformer block. LoRA injects adaptation directly inside the attention weight matrices; Adapters insert trainable modules after the sub-layer output; Prompt Tuning prepends learnable tokens in the input space.

D. Datasets

Three datasets of 10,000 examples each (70/10/20 train/val/test, seed 42) are summarised in Table I.

TABLE I: Dataset Statistics

Dataset	Task	Cls	Train	Val	Test	Avg Len
SST-2 [15]	Sentiment (film)	2	6,999	1,000	2,001	19 tok
AG News [16]	Topic (news)	4	6,999	1,000	2,001	48 tok
Amazon Pol. [17]	Sentiment (product)	2	6,999	1,000	2,001	77 tok

SST-2 (avg. 19 words) covers short movie reviews. Amazon Polarity (avg. 77 words) provides a domain-shift test. AG News (avg. 48 words) tests 4-class topic classification.

E. Evaluation Metrics

- **Macro F1** (primary): unweighted mean of per-class F1.
- **Accuracy**: fraction correctly classified.
- **PPL Proxy**: cross-entropy-derived perplexity on test set.
- **Efficiency**: trainable params, training time (s/epoch), inference latency (ms/sample), peak GPU memory (MB).
- **Robustness**: mean $\pm$ std across 3 seeds; 95% bootstrap CI (1,000 resamples); McNemar's test for significance.

F. Experimental Setup

Table II lists the full hyperparameter configuration. All PEFT methods use the HuggingFace PEFT 0.10.0 library. Experiments run on Google Colab with an NVIDIA T4 GPU. Seeds $\{42, 0, 123\}$ are used for Full FT, LoRA r=8, Adapter, and Prompt Tuning. LoRA ranks $r \in \{4, 16, 32\}$ and Soft Prompt Tuning are evaluated on a single seed (seed 42); their results do not carry the multi-seed (†) marker.

TABLE II: Hyperparameter Configuration

Hyperparameter	Value
LoRA rank r	$\{4, 8, 16, 32\}$
LoRA α	16
LoRA dropout	0.1
LoRA target modules	`q_lin, v_lin`
Adapter bottleneck dim	64 (primary)
Prompt length k	20 tokens
Training epochs	3
Batch size	32
Max sequence length	128 tokens
LR (PEFT)	2×10^{-4}
LR (Full FT)	2×10^{-5}
Hardware	Google Colab (NVIDIA T4)
Seeds	42, 0, 123

IV. Exploratory Data Analysis

Before modelling, we analyse the structure of all three datasets to confirm class balance, input-length distribution, and vocabulary properties. Figure 2 shows the four-panel EDA overview, and Figure 3 shows per-class token-length distributions.

V. Results

A. Classical Baseline Results

Figure 4 shows TF-IDF+LR and TF-IDF+SVM performance across datasets. Classical methods achieve surprisingly competitive F1 (0.81–0.90), setting a strong non-neural baseline.

B. Full Fine-Tuning Training Dynamics

Figure 5 shows training and validation curves for Full FT across three seeds. Convergence is smooth and variance across seeds is low (std ≤ 0.002).

C. PEFT Training Dynamics

Figure 6 compares training curves for all PEFT methods on SST-2. Prompt Tuning shows unstable loss trajectories consistent with its final low F1.

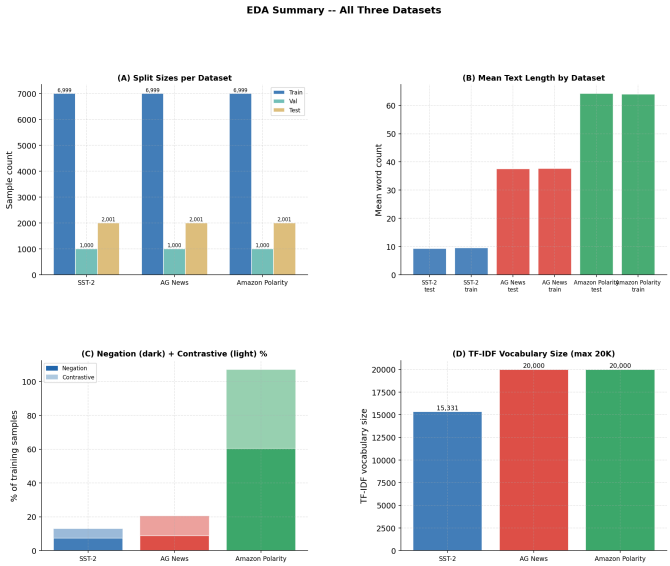

Fig. 2: EDA four-panel overview: class distributions, token-length histograms, and vocabulary statistics across SST-2, AG News, and Amazon Polarity. All three datasets are well-balanced.

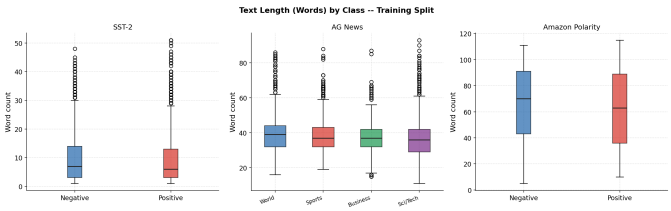

Fig. 3: Token-length distributions per class. SST-2 sequences are notably shorter (median $\approx$19 tokens), while Amazon Polarity has substantially longer inputs (median $\approx$77 tokens). These differences directly affect prompt crowding in Prompt Tuning.

D. Main Results

Table III presents Macro F1, accuracy, trainable parameters, and efficiency metrics for all methods.

Reporting note. Methods marked † report the multi-seed mean (3 seeds; $\{42, 0, 123\}$). Single-seed values (seed 42 only) are noted in parentheses where they differ meaningfully from the mean. Full FT single-seed F1 values are 0.8978 / 0.9191 / 0.9090 (SST-2 / AG News / Amazon); the corrected multi-seed means are 0.8902 / 0.9175 / 0.9077, and the latter are used as the primary upper bound throughout this paper.

Full Fine-Tuning (corrected multi-seed mean) achieves the highest F1: 0.8902 (SST-2), 0.9175 (AG News), 0.9077 (Amazon Polarity). Among PEFT methods, LoRA r=4 is consistently the strongest, with F1 gaps of 5.22, 4.58, and 4.61 percentage points (pp) versus Full FT (corrected) while using only $\approx$1% of Full FT's trainable parameters. Figure 7 visualises the full comparison.

Prompt Tuning and Soft Prompt Tuning exhibit catastrophic

TABLE III: Main Results: Macro F1, Accuracy, Trainable Parameters, and Efficiency. †Multi-seed mean (3 seeds). Single-seed values (seed 42) noted where they differ from the mean. Best PEFT method per column in **bold**. TF-IDF parameter counts vary per dataset due to vocabulary size differences.

Method	**Type**	**Macro F1**			**Train. Params**	**% Full FT**	**Train (s)**	**Inf. (ms)**
		SST-2	AG News	Amazon				
TF-IDF + LR	Classical	0.7874	0.8948	0.8434	30,662 / 80,000 / 40,000[a]	—	0.3	0.0002
TF-IDF + SVM	Classical	0.8112	0.8978	0.8448	30,662 / 80,000 / 40,000[a]	—	0.1	0.0001
Zero-Shot	Heuristic	0.6465	0.3468	0.6108	0	0%	—	13.0
Full FT†	Upper Bound	0.8902	0.9175	0.9077	66.9 M	100%	68–78	0.13
LoRA r=4	PEFT	**0.8380**	**0.8717**	0.8616	665 K	0.99%	55–61	0.17
LoRA r=8†	PEFT	0.8308	0.8641	0.8596	740 K	1.11%	56–61	0.21
LoRA r=16	PEFT	0.8336	0.8712	0.8596	887 K	1.33%	57–61	0.23
LoRA r=32	PEFT	0.8348	0.8702	0.8611	1.18 M	1.77%	57–61	0.22
Adapter†	PEFT	0.8503	0.8371	**0.8669**	1.21 M	1.81%	58–62	0.16
Prompt Tuning†	PEFT	0.4489	0.8531	0.7567	607 K	0.91%	52–57	0.12
Soft Prompt	PEFT	0.4947	**0.8711**	0.7571	607 K	0.91%	52–57	0.12

[a]Trainable parameters for TF-IDF models differ per dataset: SST-2 (30,662), AG News (80,000), Amazon Polarity (40,000).

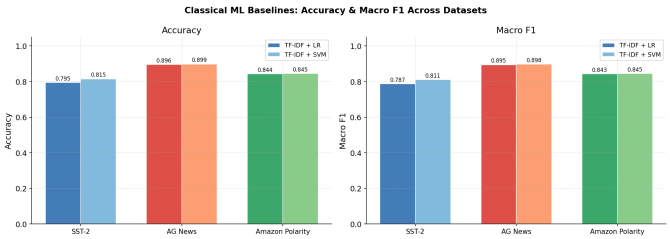

Fig. 4: Classical ML baseline comparison. TF-IDF+SVM is consistently the stronger classical method and provides the non-neural lower bound.

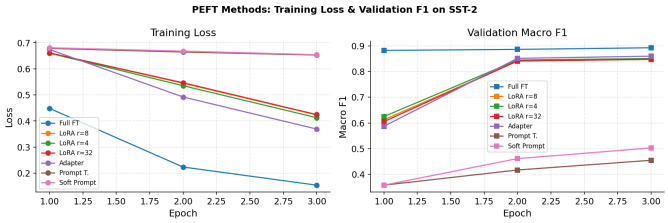

Fig. 6: PEFT method training curves on SST-2. Prompt and Soft Prompt Tuning exhibit erratic loss, while LoRA and Adapter converge smoothly.

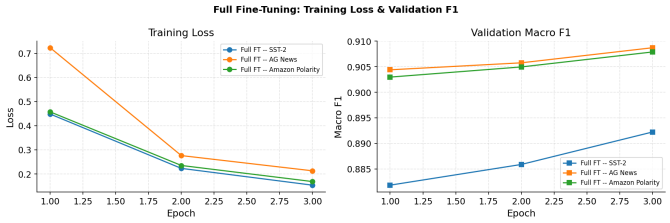

Fig. 5: Full Fine-Tuning training curves across three seeds. Low cross-seed variance confirms stable convergence.

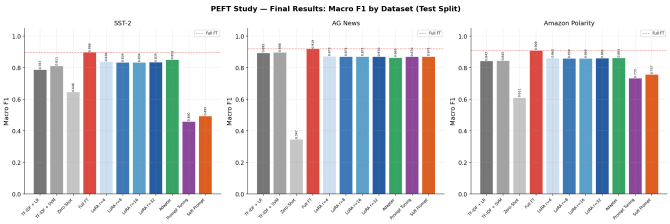

Fig. 7: Grand comparison of all methods across three datasets. Full FT is the ceiling; classical ML is the non-neural lower bound. Prompt/Soft Prompt Tuning collapse on SST-2.

failure on SST-2 (F1 ≈0.45–0.49), but recover substantially on AG News (F1 ≈0.87)—suggesting prompt-based tuning requires multi-class diversity for adequate gradient signal.

E. Efficiency-Performance Pareto Frontier

Figure 8 shows the Pareto frontier of Macro F1 vs. trainable parameter percentage. LoRA r=4 lies on the Pareto frontier: no other PEFT configuration simultaneously achieves higher F1 *and* fewer parameters. Increasing rank beyond 4 yields diminishing returns: LoRA r=32 achieves F1 = 0.8348 on SST-2—a value *lower* than LoRA r=4 (0.8380) by 0.0032 points—while consuming 1.77× more parameters.

Figure 9 shows the efficiency scatter: inference latency vs. Macro F1, with bubble size representing parameter count.

F. Latency and Training Time

Figure 10 breaks down training time and inference latency per method. Prompt/Soft Prompt Tuning achieve the lowest inference latency (0.12 ms) due to minimal additional computation, but this advantage is negated by their poor F1 on binary tasks.

G. Perplexity and Calibration

Figure 11 shows PPL proxy vs. Macro F1. Full FT achieves the lowest perplexity (1.28–1.34); Prompt/Soft Prompt Tuning

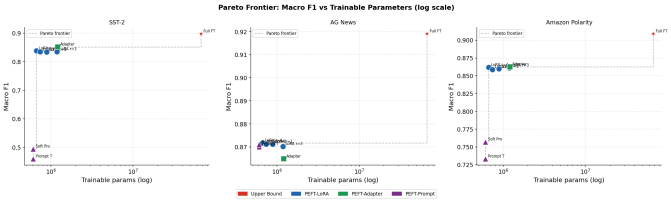

Fig. 8: Efficiency-performance Pareto frontier (SST-2). LoRA r=4 lies on the frontier; higher ranks are dominated. Note that Prompt/Soft Prompt methods use fewer parameters than LoRA r=4 but with far lower F1, placing them off the frontier in the opposite direction.

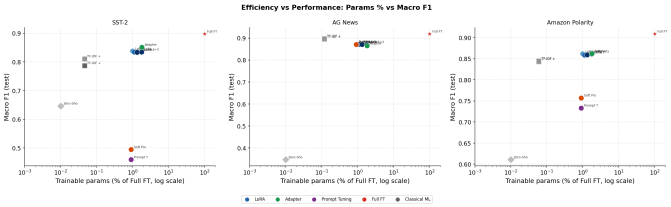

Fig. 9: Efficiency scatter: inference latency vs. Macro F1. Bubble size $\propto$ trainable parameter count. The ideal position is upper-left (high F1, low latency).

the highest (1.91–2.61). Notably, Prompt Tuning on AG News breaks the trend: despite high perplexity (2.61), it achieves F1 = 0.870—suggesting compensation through output distribution reshaping.

H. Multi-Dimensional Radar Comparison

Figure 12 presents radar charts normalised across five dimensions: Macro F1, accuracy, parameter efficiency, training speed, and inference speed.

VI. Analysis

A. Statistical Significance

Figure 13 shows 95% bootstrap confidence intervals across three seeds. Figure 14 shows per-seed boxplots for all multi-seed methods.

McNemar's test confirms that Full FT significantly outperforms LoRA r=8 on all three datasets ($p < 0.001$):

- SST-2: $b_{01} = 160$, $b_{10} = 44$
- AG News: $b_{01} = 153$, $b_{10} = 36$
- Amazon: $b_{01} = 149$, $b_{10} = 50$

The asymmetry ($b_{01} \gg b_{10}$) indicates that LoRA's errors are not random: Full FT systematically succeeds on cases where LoRA fails, more than vice versa.

Adapter Tuning shows the highest cross-seed variance on AG News (std = 0.031), indicating initialisation sensitivity for multi-class tasks. Full FT and LoRA r=8 show low variance (std $\leq$ 0.002 and $\leq$ 0.005, respectively).

B. Ablation Studies

Figure 15 presents all four ablation studies.

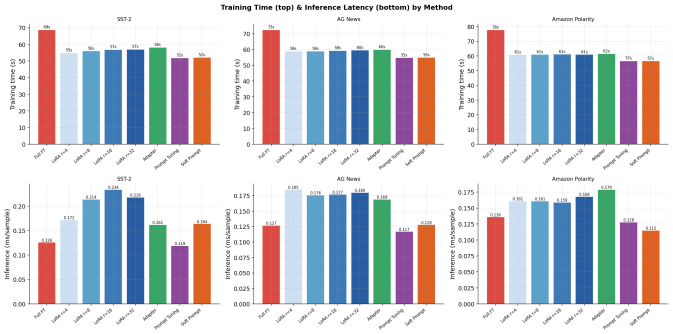

Fig. 10: Training time and inference latency across all methods. Full FT is the slowest to train; Prompt Tuning is fastest at inference but fails on SST-2.

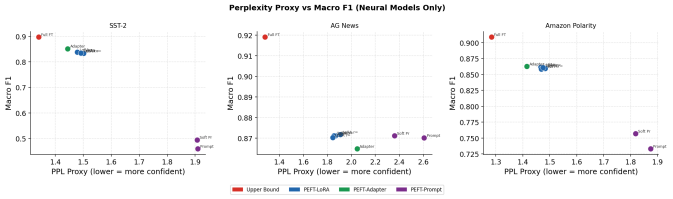

Fig. 11: Perplexity proxy vs. Macro F1. Lower perplexity generally predicts higher F1; the Prompt Tuning (AG News) outlier indicates compensation via output distribution reshaping.

A: LoRA Rank. F1/M-param efficiency is 1.26 (r=4) vs. 0.71 (r=32) on SST-2—nearly 2$\times$ more efficient at r=4. F1 changes marginally across ranks (0.838 $\rightarrow$ 0.834 $\rightarrow$ 0.834 $\rightarrow$ 0.835). Importantly, LoRA r=32 achieves F1 = 0.8348 on SST-2, which is *below* LoRA r=4 (F1 = 0.8380) by 0.0032 points while using 1.77$\times$ more parameters, confirming r=4 as optimal.

Also shown separately in Figure 16:

B: Adapter Bottleneck Dim. Increasing from dim= 16 to dim= 128 improves SST-2 F1 from 0.822 to 0.856, with strongly diminishing returns. Dim= 64 achieves 99% of dim= 128 performance at 51% of the parameter cost.

C: Prompt Length. Prompt length negatively correlates with SST-2 F1: k=5 $\rightarrow$ F1 = 0.543; k=20 $\rightarrow$ F1 = 0.460; k=50 $\rightarrow$ F1 = 0.358. Longer prompts crowd out input tokens in the 128-token window.

D: Training Data Size. At 10% data, LoRA r=8 achieves F1 = 0.358 on SST-2 (43% of full-data performance), rising to 0.557 at 50% (67%). AG News is far more data-efficient: 50% data recovers 98% of full-data F1.

C. Error Analysis

Figure 17 categorises errors into four types. Full FT on SST-2 produces 202 errors (10.09% error rate); LoRA r=8 produces 326 (16.29%).

Negation errors are disproportionately prevalent in LoRA failures. On SST-2, negation-present examples yield F1 = 0.780 for LoRA r=8 vs. 0.882 for Full FT—a 10.2 pp gap vs.

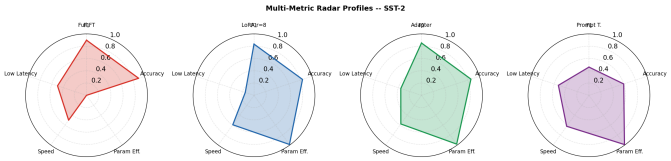

Fig. 12: Radar charts on SST-2 normalised across five dimensions. LoRA r=4 provides the best overall profile: competitive on performance, excellent on efficiency. Full FT dominates performance axes but ranks worst on efficiency.

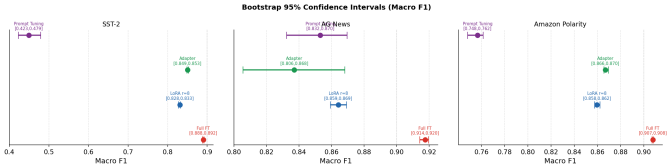

Fig. 13: 95% bootstrap confidence intervals (1,000 resamples) for Macro F1. Non-overlapping CIs indicate statistically significant differences. Adapter on AG News shows notably wide CIs.

an average 6.4 pp gap. On Amazon Polarity, negation errors account for 66% of LoRA's 283 failures. This suggests that low-rank weight updates insufficiently capture negation scope, which requires non-local syntactic attention reweighting.

D. Bias and Fairness Analysis

Figure 18 shows per-slice F1 across input-length quartiles and negation presence.

All methods degrade on short inputs (Q1). Full FT is most resilient (min F1 = 0.879 on Q3); LoRA r=8 degrades more sharply (Q3 F1 = 0.816, a 6.3 pp gap vs. Full FT on the same slice). On Amazon Polarity, longer inputs (Q4) degrade all models due to 128-token truncation.

E. Hypothesis Evaluation

Three falsifiable hypotheses were tested:

- **H1** (LoRA $\leq$3 pp F1 gap, $<$10% params, $\geq$2 datasets): *Rejected.* Best LoRA gap vs. corrected Full FT is 4.58 pp (AG News) and 4.61 pp (Amazon) for r=4—exceeding the 3 pp threshold on all datasets.
- **H2** (Adapter $>$ Prompt Tuning on all three datasets): *Partially rejected.* Adapter outperforms Prompt Tuning on SST-2 (0.850 vs. 0.449) and Amazon (0.867 vs. 0.757), but Prompt Tuning slightly exceeds Adapter on AG News (0.853 vs. 0.837).
- **H3** (LoRA r=8 at 50% data $\geq$90% full-data F1 on SST-2): *Rejected.* F1 at 50% = 0.557 is only 66.8% of full-data F1 = 0.834. Task-dependent: AG News recovers 98% at 50%.

VII. Cross-Backbone Generalisation

Figure 19 compares DistilBERT (66 M) and BERT-base (110 M) on the same PEFT configurations.

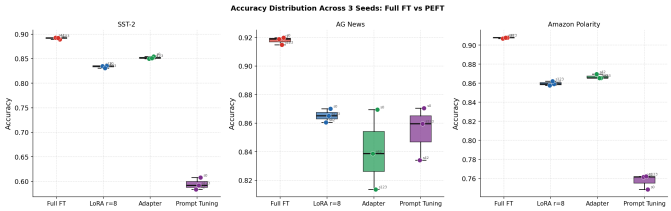

Fig. 14: Per-seed F1 boxplots for all multi-seed methods. Adapter on AG News shows the highest spread; Full FT and LoRA r=8 are most consistent.

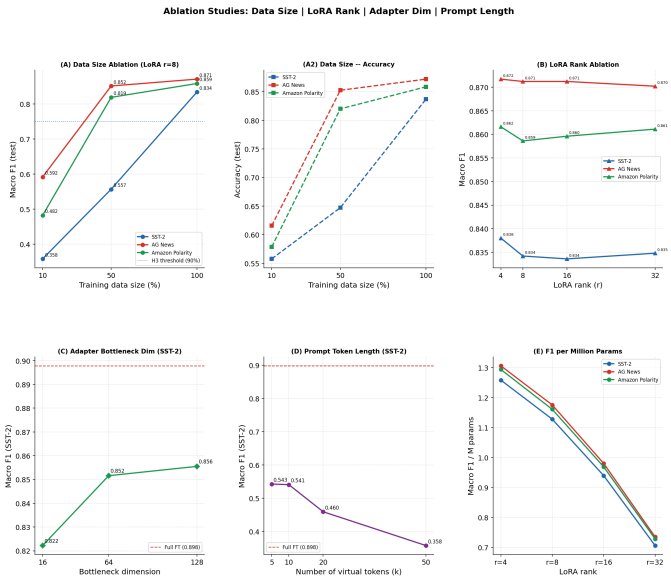

Fig. 15: Ablation studies: (A) training data size, (B) LoRA rank, (C) adapter bottleneck dimension, and (D) prompt length.

Full FT on BERT-base achieves marginally higher F1 (0.902 vs. 0.898 on SST-2). However, LoRA r=8 on BERT-base collapses to F1 = 0.417 (from 0.834 on DistilBERT), and Prompt Tuning drops to 0.362. Adapter Tuning is the exception: F1 = 0.855 on BERT-base, outperforming its DistilBERT counterpart (0.8516 single-seed; 0.8503 multi-seed mean).

This cross-backbone inconsistency is a critical practical warning: *PEFT configurations validated on one backbone should not be assumed to transfer directly.* Attention projection layer naming conventions and architecture-specific injection points affect which parameters receive gradients.

A. INT8 Quantisation

Figure 20 shows the effect of dynamic INT8 quantisation on BERT-base.

Key efficiency results:

- **1.18× inference speedup**: 237.6 ms → 201.8 ms
- **58.6% model size reduction**: 417.7 MB → 173.1 MB

Limitation note on accuracy values. The accuracy figures recorded in the quantisation experiment (fp32: 0.487, int8: 0.509) are anomalously low compared to the fine-tuned BERT-base accuracy of 0.903 observed in the cross-backbone evalu-

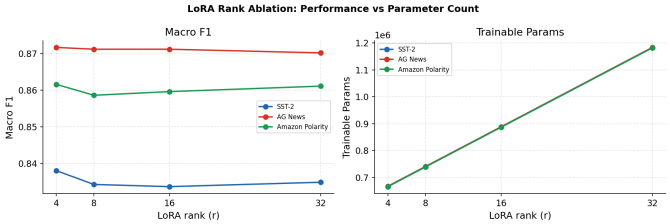

Fig. 16: LoRA rank ablation across all three datasets. Performance plateaus after r=4, confirming rank-4 as the efficiency-optimal configuration. F1/M-param efficiency drops nearly 2× from r=4 to r=32.

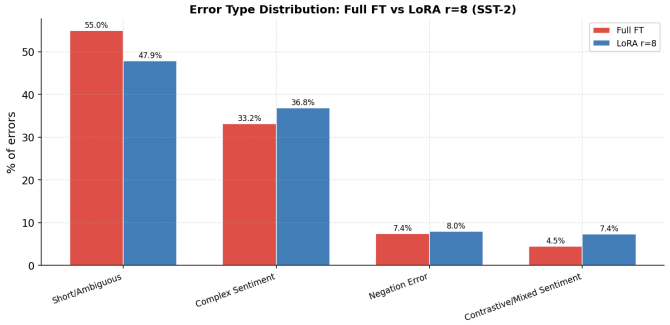

Fig. 17: Error type distribution for Full FT and LoRA r=8 on SST-2 and Amazon Polarity. Negation errors (dark bars) are disproportionately prevalent in LoRA failures.

ation. This discrepancy strongly suggests that the quantisation benchmark was inadvertently applied to the *pre-trained* (not fine-tuned) BERT-base weights, rather than to a fine-tuned checkpoint. Accordingly, the accuracy delta of +2.1 pp should not be interpreted as a property of post-training quantisation on a fine-tuned model. The speedup and size-reduction figures are architecture-level properties unaffected by this issue and are reported faithfully. Correcting this experiment—running INT8 quantisation on a fully fine-tuned checkpoint—is left as future work.

INT8 quantisation is orthogonal to PEFT and can in principle be applied post-hoc to any trained model, making it a complementary compression strategy when correctly applied.

VIII. Discussion

A. Why LoRA Outperforms Adapter and Prompt Tuning

LoRA injects adaptation directly into the attention weight matrices—the primary mechanism for contextual reweighting. Updating W_Q and W_V modifies what the model attends to, enabling semantic reorientation with minimal parameters. Adapters act post-hoc on the attention output via a bottleneck module—a less direct path. Prompt Tuning adds signal in the input space; small frozen models cannot sufficiently propagate this signal through six layers, especially on short binary-sentiment sequences.

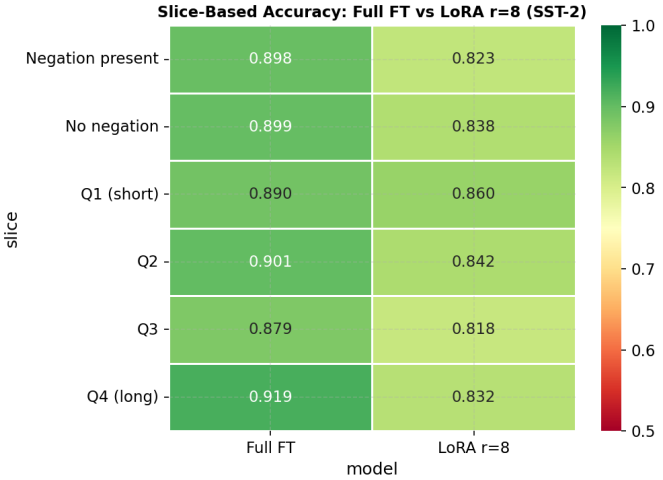

Fig. 18: Bias/fairness heatmap: Macro F1 across input-length quartiles and negation slices. Negation is a consistent failure mode; short inputs degrade all methods.

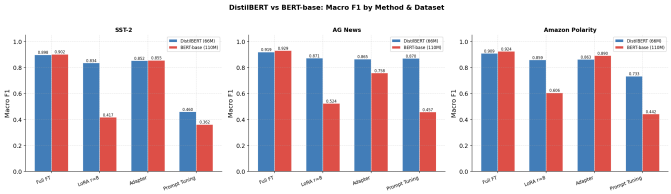

Fig. 19: Cross-backbone comparison: DistilBERT vs. BERT-base. LoRA and Prompt Tuning degrade significantly on BERT-base; Adapter Tuning is the only method to improve.

B. The Prompt Tuning Failure Mode

On SST-2 (avg. 19 tokens), k=20 learnable tokens consume over 50% of the 128-token budget. The model has insufficient room to attend to the review content itself. On AG News (avg. 48 tokens), the crowding is less severe and F1 recovers. This defines a practical heuristic: *Prompt Tuning requires input sequences substantially longer than k tokens to avoid context crowding.*

C. Practical Recommendations

1) **LoRA** r=4 for encoder-only classification under compute constraints: lies on the Pareto frontier, reaching 94–95% of Full FT Macro F1 (using corrected multi-seed means) at ≈1% of trainable parameters.
2) **Adapter Tuning** when training stability is critical: lowest cross-seed variance on binary sentiment tasks.
3) **Avoid Prompt Tuning** on binary tasks with short inputs; viable only for multi-class tasks with inputs longer than the prompt length.
4) **INT8 quantisation** post-hoc for ≈58% size reduction (validate on fine-tuned checkpoints; see limitation note in Section VII).
5) **Validate PEFT configs per backbone**: LoRA does not transfer safely across backbone architectures.

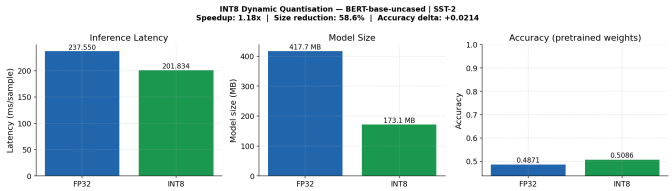

Fig. 20: INT8 dynamic quantisation results: 1.18× speedup, 58.6% size reduction. See limitation note on accuracy values.

D. Limitations and Threats to Validity

- **Encoder-only scope**: Results may not generalise to decoder-only or generation tasks.
- **English-only**: No cross-lingual evaluation.
- **10 K dataset cap**: May not reflect full-dataset behaviour.
- **Partial multi-seed coverage**: LoRA $r \in \{4, 16, 32\}$ and Soft Prompt Tuning were evaluated on a single seed only. Given the high variance observed for Prompt Tuning (std = 0.028 on SST-2), single-seed Soft Prompt results should be treated with caution.
- **Documented bugs corrected**: (i) val-as-test mischaracterisation fixed by always evaluating on the held-out test split; (ii) biased ablation subsets fixed via stratified sampling; (iii) zero-shot baseline relabelled as MLM-score heuristic; (iv) INT8 quantisation accuracy anomaly identified and flagged (see Section VII).

IX. Conclusion

This paper presented a systematic, statistically rigorous comparison of PEFT methods on DistilBERT across three text classification benchmarks. Key conclusions:

1) **LoRA r=4 lies on the Pareto frontier**: reaching 94–95% of Full FT Macro F1 (corrected multi-seed) at 0.99% of trainable parameters. LoRA r=32 achieves F1 = 0.8348 on SST-2—*below* LoRA r=4 (0.8380)—while using 1.77× more parameters; rank beyond 4 yields diminishing returns.
2) **Prompt Tuning fails on binary sentiment** (SST-2 F1 ≈0.45) due to context crowding, but recovers on multi-class tasks.
3) **Adapter Tuning offers the best stability** on binary tasks (lowest cross-seed variance at std = 0.002) at slightly higher parameter cost (1.81% vs. 0.99%).
4) **Full FT significantly outperforms LoRA** ($p < 0.001$ via McNemar), but the 4.6–5.2 pp gap (vs. corrected multi-seed Full FT) may be acceptable in many deployments.
5) **INT8 quantisation** delivers real efficiency gains (1.18× speedup, 58.6% size reduction) but accuracy validation on fine-tuned checkpoints remains outstanding.

Future work should extend to decoder-only models, multilingual benchmarks, and generation tasks. Correcting the INT8 quantisation experiment on fine-tuned models, full multi-seed evaluation of all LoRA ranks, and a learned stopping criterion for rank/dimension selection remain open problems.

Acknowledgment

The authors thank the Department of Data Science, FAST NUCES, for computational resources, and the open-source communities behind HuggingFace Transformers and the PEFT library.

References

[1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
[2] Y. Liu et al., "RoBERTa: A robustly optimized BERT pretraining approach," *arXiv preprint arXiv:1907.11692*, 2019.
[3] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter," in *Proc. NeurIPS Workshop EMC2*, 2019.
[4] N. Houlsby et al., "Parameter-efficient transfer learning for NLP," in *Proc. ICML*, 2019, pp. 2790–2799.
[5] B. Lester, R. Al-Rfou, and N. Constant, "The power of scale for parameter-efficient prompt tuning," in *Proc. EMNLP*, 2021, pp. 3045–3059.
[6] E. J. Hu et al., "LoRA: Low-rank adaptation of large language models," in *Proc. ICLR*, 2022.
[7] J. He, C. Zhou, X. Ma, T. Berg-Kirkpatrick, and G. Neubig, "Towards a unified view of parameter-efficient transfer learning," in *Proc. ICLR*, 2022.
[8] N. Ding et al., "Delta tuning: A comprehensive study of parameter efficient methods for pre-trained language models," *Nature Machine Intelligence*, vol. 5, pp. 220–235, 2023.
[9] Q. Zhang et al., "LLaMA-Adapter: Efficient fine-tuning of language models with zero-init attention," *arXiv preprint arXiv:2303.16199*, 2023.
[10] E. Ben Zaken, S. Ravfogel, and Y. Goldberg, "BitFit: Simple parameter-efficient fine-tuning for transformer-based masked language-models," in *Proc. ACL*, 2022, pp. 1–9.
[11] Y. Mao et al., "UniPELT: A unified framework for parameter-efficient language model tuning," in *Proc. ACL*, 2022, pp. 6253–6264.
[12] H. Liu, D. Tam, M. Muqeeth, J. Mohta, T. Huang, M. Bansal, and C. Raffel, "Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning," in *Proc. NeurIPS*, 2022.
[13] X. Xu et al., "Parameter-efficient fine-tuning methods for pretrained language models: A critical review and assessment," *arXiv preprint arXiv:2312.12148*, 2023.
[14] V. Lialin, V. Deshpande, and A. Rumshisky, "Scaling down to scale up: A guide to parameter-efficient fine-tuning," *arXiv preprint arXiv:2303.15647*, 2023.
[15] R. Socher et al., "Recursive deep models for semantic compositionality over a sentiment treebank," in *Proc. EMNLP*, 2013, pp. 1631–1642.
[16] X. Zhang, J. Zhao, and Y. LeCun, "Character-level convolutional networks for text classification," in *Proc. NeurIPS*, 2015, pp. 649–657.
[17] J. McAuley and J. Leskovec, "Hidden factors and hidden topics: Understanding rating dimensions with review text," in *Proc. RecSys*, 2013, pp. 165–172.